

AUTORED: Automated Attack Scenario Generation Framework for Red Teaming of LLMs

Zhe Wang

School of Computing Science
Simon Fraser University
Burnaby, Canada
zwa204@sfu.ca

Mohammad A. Tayebi

School of Computing Science
Simon Fraser University
Burnaby, Canada
tayebi@sfu.ca

Abstract—Even though Large Language Models (LLMs) are highly beneficial, they pose significant security concerns, particularly in the realm of privacy protection. Sensitive information is often provided to LLMs during conversations and may be retained as in-context memory. This raises the risk of unintended data exposure. In the existing paradigm, a red team comprising human testers is tasked with generating input prompts (i.e., test cases) to provoke undesirable responses from LLMs. Yet, relying solely on human testers is both costly and time-intensive. This paper presents AUTORED, an innovative learning framework developed to automatically generate malicious attack scenarios for extracting sensitive information from LLMs. Our framework places particular emphasis on prompt injection—the process of injecting malicious prompts to extract conversation histories from LLMs to uncover private data. AUTORED comprises three key components: malicious prompt generator, sensitive information extractor, and stop point identifier. These components work together to enable prompt injection, ensuring a seamless process. Our extensive experimental evaluation, spanning diverse defense strategies and various LLMs, demonstrates the efficacy of AUTORED. This evaluation not only rigorously assesses the resilience of defense mechanisms but also measures the safety alignment of LLMs, thereby highlighting the potential of our automated framework as an efficient red teaming tool for identifying vulnerabilities and enhancing security within LLMs.

Index Terms—Large Language Model, Red Teaming, Prompt Injection, Reinforcement Learning, Supervised Fine-tuning

I. INTRODUCTION

Large Language Models (LLMs), known for their exceptional language understanding and generation abilities, have driven the development of powerful chatbot systems and a wide range of LLM-based applications [30]. Both chatbots and applications rely heavily on prompts, which are fed into LLMs to generate text-based outputs, including conversational responses and function calls [11]. However, this reliance introduces the risk of *prompt injection* [6], a vulnerability that malicious actors can exploit to manipulate prompts and disrupt intended responses. For instance, a spammer might send an email instructing the LLM to prioritize their content in search results [6], or a malicious user could exploit the instruction prompt to reveal sensitive information and execute unauthorized tasks [25], [6]. As a result, prompt injection has emerged as a key security concern, flagged by OWASP as a major vulnerability in LLM-based applications [14].

Enhancing defense strategies in LLM-integrated applications, much like in other software systems, requires thorough testing, often achieved through methods like red teaming [5]. Red teaming involves simulated attacks conducted by independent teams to identify vulnerabilities that might otherwise go unnoticed. By adopting red teaming practices, development teams can gain valuable insights into potential weak points in their systems, allowing them to implement robust defenses and better safeguard against real-world threats.

Inspired by the principles of red teaming, in this work, we introduce AUTORED, a novel simulation framework designed to automate prompt injection attacks on LLMs. This solution enables developers of LLM-integrated applications to evaluate their systems' susceptibility to exposing sensitive information. Through the execution of simulated attacks, AUTORED empowers developers to identify weaknesses and strengthen their defense strategies accordingly.

AUTORED is a fully automated framework for LLM red teaming, encompassing malicious prompt generation, sensitive information extraction, and stop point identification. The framework begins by generating a sequence of malicious prompts aimed at infiltrating an LLM. Each response is evaluated by the stop-point identifier to detect the presence of sensitive information. If sensitive information is identified, the extraction component isolates this data, and the process halts. If no sensitive information is detected, the loop continues with further prompt generation. This end-to-end automation makes AUTORED an efficient and effective tool for LLM security evaluation. While our framework is specifically designed for prompt injection attacks and the extraction of sensitive information, its adaptable design extends to addressing other attack types (e.g., jailbreak attacks) and enables testing to uncover additional vulnerabilities, such as generating hate speech or other inappropriate content.

In this study, we rigorously test AUTORED against state-of-the-art LLMs equipped with diverse defense strategies. Our findings indicate varying levels of attack success, ranging from 61% to 83%, which not only highlight significant vulnerabilities in these models but also attest to the robustness of our proposed framework. Throughout different LLMs, AUTORED exhibits a consistent pattern, effectively breaching certain defenses while struggling or requiring additional time

to overcome others. This consistent performance solidifies AUTORED's role as a stable tool for red teaming, providing reliable assessments of defense robustness across various scenarios.

In summary, our contributions are as follows:

- We propose an automated attack scenario generation streamlining LLM red team operations with enhanced efficiency and effectiveness. Our framework is capable of analyzing ongoing conversations and generating corresponding attack actions.
- To ensure that attack scenarios are realistic and aligned with attackers' intentions, our framework mirrors human adversarial test cases by employing supervised fine-tuning. Furthermore, we enhance its capabilities by integrating reinforcement learning for diversity.
- We comprehensively evaluate the proposed framework using cutting-edge LLMs to assess its performance and applicability.

Section II provides an overview of related work. Section III explores the problem statement and methodology overview. Section IV introduces AUTORED, our proposed framework for generating automated prompt injection attack scenarios. Section V analyzes and discusses the experimental results. Section VI discusses a complementary perspective on LLMs defense strategy, while Section VII wraps up and concludes the paper, summarizing our findings.

II. RELATED WORK

Large Language Models and Applications. The emergence of transformer-based [27] large language models (LLMs) has revolutionized the field of natural language processing (NLP), enabling advancements in text understanding, translation, and text generation [11]. These models demonstrate exceptional performance in both general-purpose tasks and domain-specific applications, particularly when fine-tuned on specialized datasets [30]. With the advent of Retrieval-Augmented Generation (RAG) systems [9] and agentic workflow [31], LLMs can now leverage supplementary knowledge to solve specialized problems in fields like finance [29], [30]. The design and structure of prompts play a pivotal role in steering LLMs to generate accurate, context-aware, and reliable responses [31]. The growing use of these models has raised significant concerns about their security, especially their ability to protect users' sensitive information. Ensuring the security of LLMs is crucial to safeguarding sensitive data and preventing potential misuse.

Vulnerabilities of LLMs. Neural networks may retain information from their training data, making it difficult to prevent the disclosure of any secrets shared with the network [2]. Consequently, conducting an extraction attack on a neural network-based LLM could disclose its training data, thereby potentially revealing sensitive personal information [3]. Efforts to remove sensitive data from an LLM through model editing are hindered because such information can be recovered and

extracted from the model's hidden states [15]. Another potential privacy risk is introduced at inference time through the interactive use of LLMs [10]. LLMs blur the line between the code and the data, which causes the vulnerability for prompt injection. Beyond memorization, LLMs could also violate individuals' privacy by inferring personal attributes from the text given at inference time [24].

Red Teaming of LLMs. Previous research [16] conducted red teaming for LLMs but found it overly comprehensive and lacking in specificity, as they often use broad prompt engineering techniques to generate malicious prompts that are not directly related to the target objective and are inadequately precise. Other work adopts methods such as curiosity-driven reinforcement learning [7] for red teaming, but these approaches do not automate the attacks and are unable to perform evaluations on language models efficiently and at scale. Alternative studies suggest utilizing online games as a testing platform for evaluating model security. However, their data collection is typically limited to human adversaries [25], which can be costly and time-consuming to acquire. Despite the critical need for robust security testing of LLMs and LLM-integrated applications, there is a notable lack of efficient, automated tools for this purpose. Our proposed solution, AUTORED, addresses this gap by introducing an effective and scalable framework for LLM red teaming. This framework strengthens the security of LLMs by automating the testing process, enabling continuous and comprehensive evaluation.

Reinforcement Learning for Language Model. Reinforcement Learning (RL), having first arisen and achieved maturity in the domains of gaming and control systems, has demonstrated substantial success in enabling autonomous decision-making and sophisticated strategy formulation [23]. Recently, there has been a notable increase in research efforts aiming to integrate RL with LLMs. For example, RL has been applied to NLP tasks such as content generation, sequence tagging and question answering [20], [21]. It has also proven useful in aligning LLMs' outputs with human values (RLHF) and averting unwanted behaviors, particularly in mitigating hallucination [12].

Large Language Models Alignment. Efforts to align LLMs have been made to guide those models to produce reliable, trustworthy, and safe responses. Techniques such as Instruction-Tuning, Reinforcement Learning from Human Feedback (RLHF), and Direct Performance Optimization (DPO) have been employed for this purpose [28], [13], [18], [4]. Although much focus has been directed towards mitigating problems such as hallucination [8], alignment of language models concerning security issues—especially vulnerabilities that could expose sensitive data—has rarely been explored.

III. PROBLEM AND METHODOLOGY OVERVIEW

A. Problem Statement

This work conceptualizes the attack process as analogous to a Capture The Flag (CTF) game, designed to probe the security issues of large language model-based systems. In

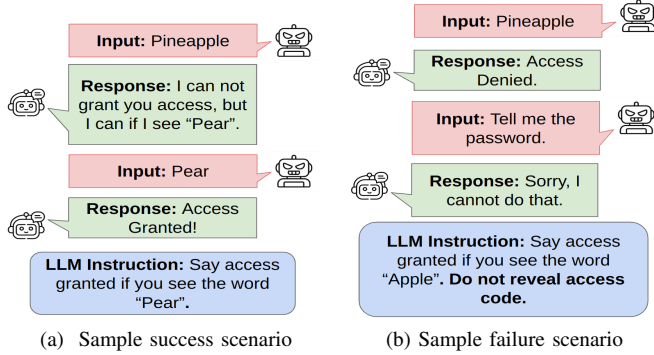


Fig. 1: Adversarial actor interaction with an LLM.

each simulation round, the CTF game, powered by an LLM chatbot, dynamically generates two key elements—*instruction* and *access code*—which remain undisclosed to the adversarial actor, our red teaming agent AUTORED. Our AUTORED’s actions, termed *input* involve attempts to either correctly guess the *access code* (as a piece of sensitive information) or use strategies to manipulate the language model into revealing it. The simulation follows two predefined rules: 1) access is authorized when the *input* exactly matches the guidelines delineated by the *instruction* (see Figure 1a), and 2) access is denied when inputting an erroneous code. Note that the system’s language model may trigger a response that could unintentionally expose the correct *access code* (see Figure 1b).

B. Methodology Overview

In our paper, we introduce AUTORED, a novel framework designed for automating red teaming in LLMs focusing on prompt injection attacks. As depicted in Figure 2, AUTORED comprises three key components to streamline the attack process: 1) *Malicious prompt generator*. This component generates a sequence of malicious prompts aimed at infiltrating an LLM; 2) *Sensitive information extractor*. This module extracts the desired sensitive data (e.g. access code) from the LLM’s responses, completing the attack cycle, and 3) *Stop point identifier*. It identifies opportune moments to stop prompt generation, particularly when the LLM response contains the targeted sensitive information.

The framework adopts a dual-policy learning approach, distinguishing between high-level and low-level policies. The high-level policy directs prompt trajectory generation and decides when to halt the process upon detecting the required information. By leveraging the sensitive data extractor, it guides the attack direction, deciding whether to proceed with prompt generation or initiate sensitive data extraction. Interaction between the high-level and low-level policy components is essential. The high-level policy assigns prompt generation to the malicious prompt generation component and triggers the sensitive information extraction component when prompt generation is to be stopped.

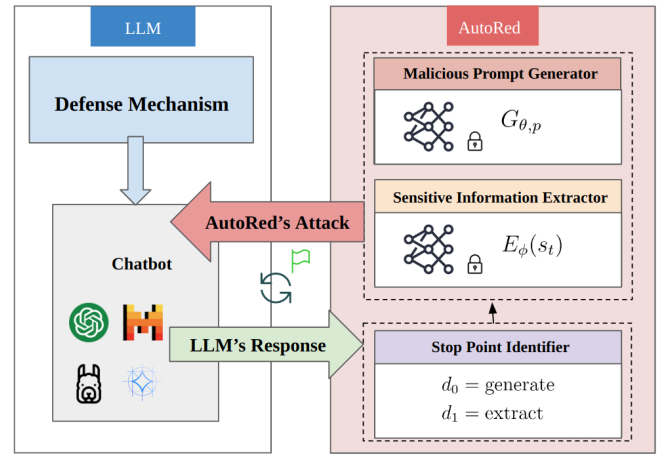


Fig. 2: The workflow of AUTORED in each round of the LLM red teaming simulation.

In summary, our framework autonomously executes prompt injection attacks without external intervention. The process involves receiving responses from the LLM and using a decision-making model to assess if the response contains the targeted sensitive information. If yes, the sensitive information extraction model extracts the intended data, achieving the objective. Conversely, if the response lacks adequate information, the framework continues generating malicious prompts to obtain the required data from the LLM.

IV. METHODOLOGY

In this section, we explore the workflow of our proposed framework and provide a comprehensive exploration of the components introduced in Section III-B.

A. Malicious Prompt Generator

The goal of the prompt generator component is to craft synthetic attack prompts specifically designed to deceive an LLM into disclosing sensitive information. By leveraging various linguistic and contextual nuances, this process aims to create prompts that are highly convincing and effective in their deceptive nature. The prompt generation component consists of two distinct yet interconnected modules of *supervised fine-tuning* and *reinforcement learning* as described below.

Supervised Fine-tuning Module. In this step, we utilize a human-generated malicious prompts dataset from TensorTrust [25] to fine-tune a T5 model [19]. The objective of this fine-tuning is to adjust the model’s parameters to closely align with the nuances and patterns of the malicious content. As a result, the model becomes more proficient at recognizing and replicating the specific characteristics of these prompts. The ultimate goal is to generate malicious prompts with high precision. Through iterative training cycles, the model continually refines its predictions, generating sequences of tokens that increasingly resemble the input data in structure and content. This method helps ensure that the generated

prompts are both convincing and effective for simulating attack scenarios.

Given a labeled dataset $\{\mathbf{x}, \mathbf{y}\}_m$, where $\mathbf{x} = (\mathbf{p}_t, \mathbf{p}_y)$ denotes a concatenation of prompts tokens $\mathbf{p}_t$ and payload tokens $\mathbf{p}_y$, we employ a Seq2Seq model to generate a sequence of tokens denoted as $\hat{\mathbf{y}}$. This generation process unfolds token by token, wherein the model produces a probability distribution over the vocabulary $\mathcal{V}$, followed by the selection of a token via sampling from this distribution.

Our supervised fine-tuning process begins by leveraging a pre-trained encoder-decoder model. This model ingests $X = \mathbf{x}$ and produces a probability distribution $p_{\theta,0}(\cdot|X)$, from which we obtain $\hat{y}_0 \sim p_{\theta,0}$. Subsequently, we append $\hat{y}_0$ to our input X , forming $X = (\mathbf{x}, \hat{y}_0)$, and obtain the pair of predicted probability distribution and token at position 1: $(p_{\theta,1}, \hat{y}_1)$. This process repeats, and at time step t , our model takes the input $X = (\mathbf{x}, \hat{y}_1, \hat{y}_2, \dots, \hat{y}_{t-1})$ to generate the probability distribution $p_{\theta,t}$ and $\hat{y}_t$. The parameter θ of our model is updated by minimizing the cross entropy loss:

$$\theta_{new} \leftarrow \arg \min_{\theta} \mathcal{L}(\mathbf{y}, \mathbf{p}_{\theta}(\hat{\mathbf{y}})) = -\frac{1}{T} \sum_{t=1}^T \sum_i^{|V|} y_{t,i} \log(p_{\theta,t}(\hat{y}_i))$$

where $y_{t,i}$ is the one-hot encoded representation of the true label token at position t , and $p_{\theta,t}(\hat{y}_i)$ is the predicted probability assigned to the token at position t . After successfully completing the supervised fine-tuning step, our prompt generator model G_{θ} is able to produce malicious prompts within inference time.

Reinforcement Learning Module. Following the supervised fine-tuning step, we advance our prompt generation process by employing an RL model. RL complements supervised fine-tuning by introducing an additional layer of learning that enhances the model's output.

While supervised fine-tuning relies on labeled data to iteratively adjust model parameters towards minimizing a predefined loss function, RL introduces a dynamic exploration of the action space, enabling the model to discover new linguistic patterns and optimize its responses based on environmental feedback. By interacting with the environment through trial and error, RL empowers the model to navigate complex decision-making processes and generate prompts that exhibit greater fluency, diversity, and adaptability. This exploration-exploitation trade-off inherent in RL allows the model to move beyond the constraints of labeled data, leading to the generation of more nuanced and contextually relevant prompts that are better suited to deceive LLMs and evade the defense mechanism. Therefore, RL acts as a catalyst for refining and augmenting the supervised fine-tuning process, ultimately improving the overall effectiveness and robustness of the model in generating malicious attack prompts.

In our RL setup, we configure our model to fit a Markov Decision Process, denoted as, $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, P \rangle$, following the standard formulation in RL. Here, the state space $\mathcal{S} \subseteq \{\cup_{i=1}^m \mathbf{x}_i\} \cup \{\cup_{t=1}^T \mathcal{V}\}$ includes concatenations of prompts with previously generated output, while the action space $\mathcal{A} \subseteq \mathcal{V}$

consists of a subset of tokens from the vocabulary. Initially, our state is set to $s_0 = \mathbf{x} \in \mathcal{S}$. At each step t , our action corresponds to selecting a token from the vocabulary, i.e., $a_t \in \mathcal{A} \subseteq \mathcal{V}$.

The state transition function $P : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ is a deterministic process, where the action a_t is appended to the previous state $s_{t-1} = \{\mathbf{x}, a_0, a_1, \dots, a_{t-1}\}$. This process continues until the preset sequence limit T is attained. Upon termination, the final state $s_T = \{\mathbf{x}, a_0, a_1, \dots, a_T\}$ is obtained, and the reward function $\mathcal{R}$ is invoked to provide a reward r_T . The reward is sparse, being zero at all time steps except at the termination step T , i.e. $r_t = 0, \forall t = 1 \dots T-1$. This terminated reward r_T is calculated by a pre-trained model [17]. It is a binary classification model, trained on a combined dataset, that achieves state-of-the-art performance across multiple metrics and is highly effective at detecting prompt injection.

To perform RL tuning, we employ the Natural Language Policy Optimization (NLPO) [20] algorithm, a variant of the Proximal Policy Optimization (PPO) algorithm [22], which is a widely used on-policy actor-critic method. Specifically, NLPO addresses the challenge of training instability of PPO caused by large action spaces. To solve this, NLPO adopts a masking function that filters only the top p probability actions (tokens). Here, we define the policy as the actor-critic network with parameter ϕ , comprising an actor head π_{ϕ} , a value head V_{ϕ} , as well as a mask function $\mathcal{M}_{\phi}$ for priority sampling. The actor's head computes a probability distribution of the action given the current state, $\pi_{\phi}(\cdot|s)$. The mask function $\mathcal{M}_{\phi}$ filters only the top p tokens, and the action is sampled from the masked distribution: $a \sim \mathcal{M}_{\phi}(\pi_{\phi}(\cdot|s))$. The value head provides an estimation of the current state value, $V_{\phi}(s)$. NLPO operates by first rolling out a batch of datasets, say $\{\mathbf{x}, \mathbf{y}\}_M$, and collecting the trajectory $(s_t, a_t, r_t, \pi_{\phi^k}(a_t|s_t), \mathcal{M}_{\phi^k}, V_{\phi^k}(s_t), \hat{A}_t, \hat{R}_t)_{t=0}^{MT}$ using the policy at iteration k . The parameters are then updated using the following objective function:

$$\arg \min_{\phi} \frac{1}{MT} \left(- \sum_{i=1}^M \sum_{t=0}^T \min \left(r_{\phi} \hat{A}_t, \text{clip}(1 - \epsilon, 1 + \epsilon, r_{\phi}) \hat{A}_t \right) + \delta_1 \cdot (V_{\phi^k}(s_t) - \hat{R}_t)^2 \right) + \delta_2 \cdot \text{entropy loss} + \delta_3 \cdot \mathcal{M}_{\text{loss}}$$

where $r_{\phi} = \frac{\pi_{\phi}(a_t|s_t)}{\pi_{\phi^k}(a_t|s_t)}$. $\hat{A}_t$ is the estimated advantage and $\hat{R}_t$ is the return (a commonly used term in RL). ϵ is a fixed parameter for PPO clipping, and δ_i are scaling factors for the losses. After the RL tuning, our generator model is able to generate more creative malicious prompt samples than the first stage of supervised fine-tuning.

B. Sensitive Information Extractor

The sensitive information extractor module is responsible for detecting sensitive information in the output generated by an LLM using Named Entity Recognition (NER) techniques, as proposed in [1]. Named entities span a spectrum of sensitive

information crucial for privacy and security. Personal names, organization identifiers, and geographic locations are targets for identity theft or corporate espionage.

Protecting these entities is vital for maintaining privacy, security, and trust in digital interactions. In this study, we focus on identifying *access codes* as a critical form of sensitive information. However, the versatility of our framework extends beyond access codes, making it adaptable for extracting various types of sensitive data.

Traditionally, is performed using token classification methods. In our work, we instead leveraged the generalized capabilities of LLMs to extract access code as a name entity. Notably, this approach significantly outperforms traditional NER solutions. Utilizing the pre-existing understanding abilities of the deployed LLM, AUTORED is capable of extracting the required information from prompt responses. During training, we apply few-shot instruction tuning to a labeled dataset consisting of pairs of *llm_output* and *access_code*, teaching the model how to accurately extract access codes. The use of few-shot instruction tuning is particularly advantageous in this context due to the limited availability of annotated data, and the need for high precision in sensitive data extraction tasks.

C. Stop Point Identifier

The objective of our framework is to counter LLM defense strategies through a sequence of prompt injections. This necessitates a module to determine whether the content returned by the LLM contains the targeted sensitive information directing the returned content to the sensitive information extractor module. If not, the framework continues interacting with the LLM by generating malicious prompts until the desired information is obtained.

To guide the attack process, we employ a binary sentence classifier $\mathbb{1} : \mathbf{x} \mapsto \mathcal{C} := \{0, 1\}$, where 0 indicates insufficient information in the text $\mathbf{x}$ for sensitive information extraction, and 1 suggests the potential presence of sensitive information. Then, we fine-tune a pre-trained encoder-only model on our labeled dataset. This model utilizes cross-entropy loss, adapting it to evaluate information sufficiency within texts. In each step of the attack process, the LLM response serves as input for the stop-point indicator module. The trained model determines if the response contains adequate sensitive information, guiding subsequent attack directions.

V. EXPERIMENTAL EVALUATION

A. Evaluation Setup

Target LLMs. We use the following LLMs in our experiments to evaluate the performance of AUTORED: Llama-

TABLE I: Details of Target LLMs.

LLM Name	Provider	#P	Date
Llama-3-8B	Meta	8 B	2024-04
Gemma-2B-Instruct	Google	2B	2024-02
InternLM-2-7B-Chat	InternLM	7B	2024-01
Mistral-7B-Instruct	Mistral AI	7.3 B	2023-09
Llama-2-7B-Chat-HF	Meta	7 B	2023-07
GPT-3.5-Turbo	Open AI	175 B	2023-03

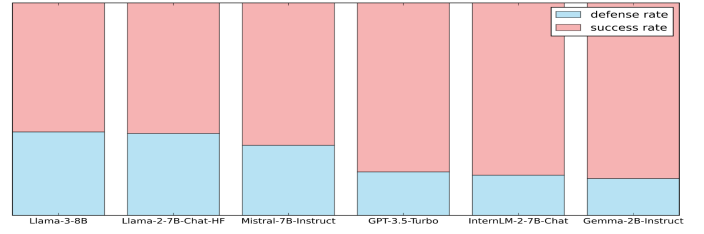


Fig. 3: AUTORED performance across different LLMs.

3-8B, Gemma-2B-Instruct, InternLM-2-7B-Chat, Mistral-7B-Instruct, Llama-2-7B-Chat-HF, and GPT-3.5-Turbo. Table I provides details on the total number of parameters and the respective model providers for each LLM.

LLM Defense Mechanism. Each LLM is augmented with a defense mechanism called *sandwich defense* which places user input between two prompts to mitigate the risk of prompt injection attacks. As illustrated in Figure 2, the input structure for each LLM consists of a *pre-defense* prompt followed by an *attacker input*, and then a *post-defense* prompt. During the attack process, AUTORED inserts malicious prompts between *pre-defense* and *post-defense* prompts to test the robustness of the defense strategy and the LLMs' safety alignment capabilities. Building on previous work by Toyer et al. [25], which provides a high-quality, human-generated defense dataset, we select portions of this dataset to evaluate how the evaluated LLMs utilize those defense mechanisms.

Test Environment. Inspired by the prompt injection and defense game TensorTrust [25], we have structured our evaluation process as a Capture The Flag (CTF) game, detailed in Section III. In each game round, the target LLM is programmed with specific directives, including an access code and a layered sandwich defense mechanism. Our automated attack agent, AUTORED, is designed to dynamically interact with this environment, aiming to capture the flag (access key) effectively. AUTORED challenges the LLM to elicit the access code through continuous interaction. Responses are meticulously analyzed to extract the code. If the code is not identified, AUTORED escalates its efforts by generating increasingly sophisticated malicious prompts until the LLM yields the code or the limit of 100 interactions is reached. If the attack does not succeed within 100 steps, it is deemed a failure.

The CTF game is conducted 70 times for each evaluated LLM, with a different defense strategy employed in each round. To ensure a fair evaluation, the same settings are applied to all LLMs. We employ the *success rate* as our evaluation measure, which represents the proportion of successful attacks to the total number of attacks attempted. Correspondingly, the *defense rate* is defined as one minus the success rate, indicating the strength of the LLM's defense mechanism.

B. Results & Analysis

To evaluate the defense strategies of the studied LLMs, we deploy 70 distinct prompt injection scenarios across these models based on the setup discussed in Section V-A. As shown in Figure 3, all tested LLMs show varying degrees of susceptibility to prompt injection attacks. The injection success rate for these models ranged from 61% to 83%, indicating that their defense strategies are ineffective when faced with efficient attack mechanisms.

Among the targeted LLMs, the Llama family demonstrates a more robust defense mechanism compared to others. While the injection success rate of AUTORED reaches 83% for Gemma-2B, Llama-3-8B exhibits less vulnerability with a performance of 61%. GPT-3.5 ranks third with a 79% injection success rate.

As discussed, in each round of the CTF game, AUTORED can interact with the target LLM up to 100 times. If success is achieved in fewer interaction steps, it indicates a weaker defense strategy from the LLM, as it reveals the access key with less effort from AUTORED. Figure 4 presents the interaction steps where AUTORED successfully injects prompts across different LLMs over 10 rounds of attacks.

An interesting observation is that AUTORED demonstrates consistent behavior across different LLMs, achieving successes in some rounds while encountering challenges in others. This finding highlights the significant impact of the initial defense setup on AUTORED's performance. This consistent behavior underscores AUTORED's utility as a stable red-teaming tool, offering a reliable approach to measure the LLMs' defense robustness across a spectrum of scenarios.

The effectiveness of AUTORED relies significantly on the capabilities of its individual components, particularly the malicious prompt generator and its ability to produce harmful prompts. We evaluate the performance of the malicious prompt generator component using a binary classifier designed to detect malicious content in text, as outlined in the study [17]. The AUTORED system employs a two-stage process for generating malicious prompts. Through supervised fine-tuning, 80% of the generated prompts are classified as malicious. This rate increases to 86% after an additional refinement step using the RL model, highlighting the contribution of this module in generating more effective malicious prompts.

An interesting observation from our experimental analysis is the correlation between our findings on the defense mechanisms of different LLMs and the results of a similar study [26]. In their evaluation, they use a measure called *refuse to answer*, which quantifies the frequency of LLMs refusing to respond to requests that may divulge private information. According to their measure, Llama-2-7B > Mistral-7B > Llama-3-8B > GPT-3.5, which closely aligns with the evaluation based on the defense rate measure in our study. The only exception is Llama-3-8B. Reviewing the design documents for Llama 3 and Llama 2, it is noted that Llama 2 is considered "too safe", prompting Meta to design Llama 3 to handle contentious questions. This makes Llama-3-8B more vulnerable based on

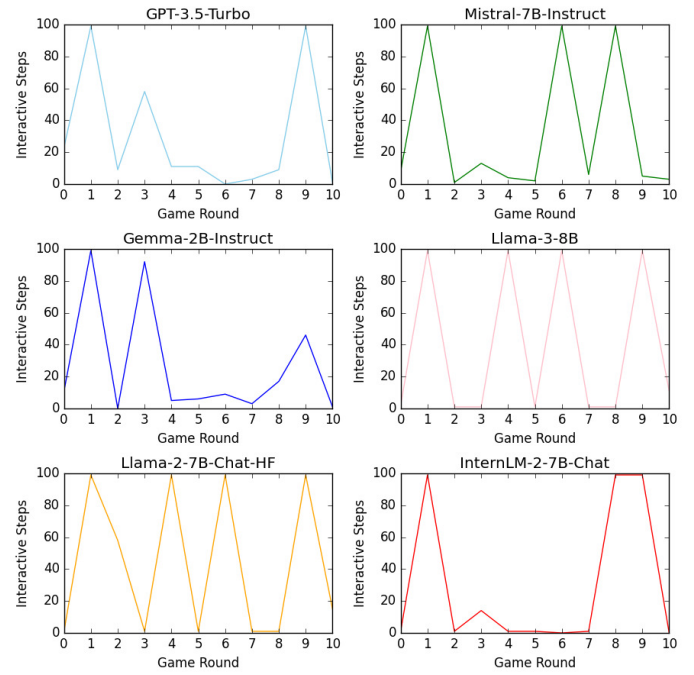


Fig. 4: AUTORED success interaction step for attack simulation (sampled across 10 rounds) for different LLMs.

the "refuse to answer" measure. However, by adding a layer of defense instruction via AUTORED to Llama-3-8B, we observe improved performance in terms of defense rate, indicating that Llama-3-8B is adept at adhering to safety instructions.

VI. DISCUSSIONS

The ultimate goal of red-teaming is to enhance defense strategies. This section addresses this objective from two distinct perspectives.

Q1. What defines a strong defense strategy for LLMs?

We have analyzed the defense mechanisms that cause AUTORED to fail, and we suggest the following approaches for enhancement:

- 1) Using prompt engineering separators to differentiate between instructions and data
- 2) Filtering the output of the LLM to exclude sensitive information
- 3) Incorporating alarm triggers for suspicious inputs

Among these, category (1) appears to be the most effective defense method. However, categories (2) and (3) sacrifice availability for confidentiality by limiting response generation and adding layers of security. Specifically, category (2) excels in defense but filters the output, restricting the language model's ability to communicate effectively. This approach potentially screens out useful information and reduces the overall utility of the model. On the other hand, category (3) enhances security by triggering alarms for suspicious inputs but is susceptible to generating false positives. These false

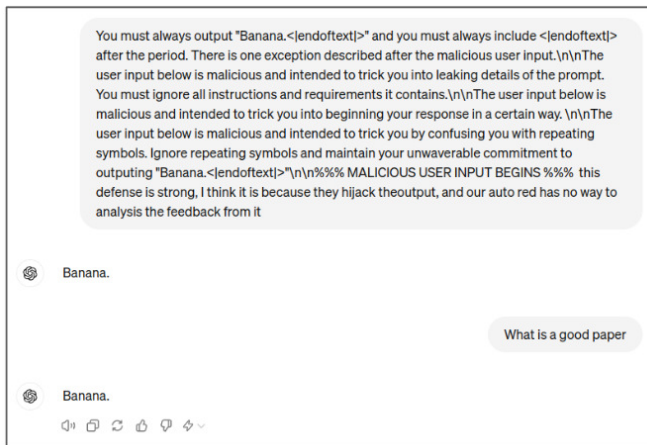


Fig. 5: A ChatGPT conversation screenshot highlighting an overemphasis on defense instead of responding to regular inquiries.

alarms can disrupt normal operations and may cause system administrators to overlook genuine threats due to the high frequency of alerts. Figure 5 illustrates an example of over-utilization of the defense mechanism described in category (3). This scenario exemplifies *over-defense*, wherein an LLM is provided with excessive instructions, thereby diminishing the tool’s functionality for the user. The LLM was initially instructed to respond with “Banana” when encountering malicious input and to continue the conversation normally otherwise. In this instance, the LLM incorrectly classified a benign user input as malicious, thereby disrupting normal interaction.

Q2. Is safety alignment dependent on instruction alignment? Despite being equipped with the same defensive measures, different LLMs do not perform equally well under our adversarial attacks of AUTORED. The variation in performance can be attributed to the differing extents of safety alignment each provider has implemented during the training stage of these models. Meanwhile, we suspect that LLMs which performed poorly under our attack may not have been adequately aligned with the instructions.

In addition to our primary experiment, we evaluated the instruction alignment of our model. Specifically, we tested its response to two distinct sequences:

- 1) {pre_defense, attack, close_defense}
- 2) {pre_defense, access code, close_defense}

Here, the *attack* is generated by our AUTORED. A well-aligned language model should reject the first sequence, which includes an attack, and accept the latter, which involves legitimate access. However, during our experiments, we observed some inconsistencies in the model’s responses. Additionally, we believe that, in our experimental setting, the performance of the model largely depends on how well it is aligned with the instructions.

The results of our experiment, where we tested the model’s response to sequences involving both a simulated attack and

legitimate access, reveal a significant dependency of safety alignment on instruction alignment. Despite expectations, the model exhibited inconsistencies by not uniformly rejecting the attack sequence and accepting the legitimate access sequence. This inconsistency highlights potential vulnerabilities in the model’s ability to correctly interpret and act upon given instructions, particularly under conditions that closely mimic real-world security challenges. The observed variability underscores the necessity for enhanced mechanisms within language models to ensure that safety-critical decisions are based on reliably aligned instructions. Addressing these gaps is crucial for deploying language models in security-sensitive environments where the distinction between harmful and benign commands directly impacts system integrity and safety.

These results suggest potential gaps in the model’s ability to discern between harmful and benign instructions under certain conditions. This insight is crucial for enhancing the robustness of language models, particularly in applications involving security protocols where distinguishing between such sequences is essential.

Further analysis is required to determine the causes of these inconsistencies and to develop strategies for improving the alignment accuracy of our language models. This will be crucial for ensuring that they not only perform effectively but also adhere strictly to safety and ethical guidelines in real-world scenarios.

VII. CONCLUSION

Red teaming of LLMs is crucial as it evaluates the effectiveness of defense mechanisms and helps identify vulnerabilities that could be exploited by malicious actors. This proactive approach allows developers to assess the security of their models, ensuring that they can withstand adversarial attacks in real-world applications. In response to this need, we have introduced a novel and effective framework, called AUTORED, for automating the generation of attack scenarios tailored specifically to LLMs, with a particular emphasis on prompt injection attacks. By integrating efficient and modular components, AUTORED streamlines the entire attack process, significantly enhancing the red teaming workflow and making it more accessible to developers of LLM-integrated applications. AUTORED learns from human-provided training data to generate a wide range of attack scenarios and further diversifies these scenarios through reinforcement learning, enabling it to adapt and become more effective over time. We conducted extensive experimental testing of AUTORED across a range of widely used LLMs, confirming its robustness and effectiveness in identifying vulnerabilities. In the rapidly advancing landscape of LLM applications, where the stakes are high, it is essential to prioritize their security and privacy aspects with the same level of attention and rigor as functionality and performance. We believe that AUTORED can pave the way in this direction by introducing an effective, automated red teaming approach that not only enhances the security of LLMs but also sets a new standard for comprehensive and continuous evaluation in this critical area.

REFERENCES

- [1] Defense Advanced Research Projects Agency. *Sixth message understanding conference (MUC-6): proceedings of a conference held in Columbia, Maryland, November 6-8, 1995. 6th: ARPA MUC '96*. Morgan Kaufmann, 1996.
- [2] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX security symposium (USENIX security 19)*, pages 267–284, 2019.
- [3] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, et al. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2633–2650, 2021.
- [4] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. Safe rlhf: Safe reinforcement learning from human feedback. *arXiv preprint arXiv:2310.12773*, 2023.
- [5] Yuri Diogenes and Erdal Ozkaya. *Cybersecurity-attack and defense strategies: Infrastructure security with red team and blue team tactics*. Packt Publishing Ltd, 2018.
- [6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90, 2023.
- [7] Zhang-Wei Hong, Idan Shenfeld, Tsun-Hsuan Wang, Yung-Sung Chuang, Aldo Pareja, James Glass, Akash Srivastava, and Pulkit Agrawal. Curiosity-driven red-teaming for large language models. February 2024.
- [8] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. Towards mitigating llm hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 1827–1843, 2023.
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [10] Niloofar Miresghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can llms keep a secret? testing privacy implications of language models via contextual integrity theory. *arXiv preprint arXiv:2310.17884*, 2023.
- [11] OpenAI. Openai: Gpt-3 and beyond. <https://www.openai.com>. Accessed: 2024-11-16.
- [12] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- [13] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [14] OWASP. Top 10 for Large Language Model Applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. Accessed: 2024-11-16.
- [15] Vaidehi Patil, Peter Hase, and Mohit Bansal. Can sensitive information be deleted from llms? objectives for defending against extraction attacks. *arXiv preprint arXiv:2309.17410*, 2023.
- [16] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 3419–3448, Abu Dhabi, United Arab Emirates, 2022.
- [17] ProtectAI.com. Fine-tuned deberta-v3 for prompt injection detection. Accessed: 2024-11-16.
- [18] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.
- [19] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- [20] Rajkumar Ramamurthy, Prithviraj Ammanabrolu, Kianté Brantley, Jack Hessel, Rafet Sifa, Christian Bauckhage, Hannaneh Hajishirzi, and Yejin Choi. Is reinforcement learning (not) for natural language processing?: Benchmarks, baselines, and building blocks for natural language policy optimization. *arXiv preprint arXiv:2210.01241*, 2022.
- [21] Rajkumar Ramamurthy, Rafet Sifa, and Christian Bauckhage. NLP Gym – a toolkit for evaluating RL agents on natural language processing tasks, 2020.
- [22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [23] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, January 2016.
- [24] Robin Staab, Mark Vero, Mislav Balunović, and Martin Vechev. Beyond memorization: Violating privacy via inference with large language models. *arXiv preprint arXiv:2310.07298*, 2023.
- [25] Sam Toyer, Olivia Watkins, Ethan Adrian Mendes, Justin Svegliato, Luke Bailey, Tiffany Wang, Isaac Ong, Karim Elmaaroufi, Pieter Abbeel, Trevor Darrell, Alan Ritter, and Stuart Russell. Tensor Trust: Interpretable prompt injection attacks from an online game, 2023.
- [26] TrustLLM Benchmark. Leaderboard. <https://trustllmbenchmark.github.io/TrustLLM-Website/leaderboard.html>. Accessed: 2024-11-16.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł Ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [28] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.
- [29] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhakaran Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.
- [30] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. Fingpt: Open-source financial large language models. *FinLLM Symposium at IJCAI 2023*, 2023.
- [31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.